



Who fear the saga?

A saga = a long story of heroic achievement

github.com/BrewUp/KanDDDisky-2024

Goals

- Understand sagas
- Build a saga
- Test a saga



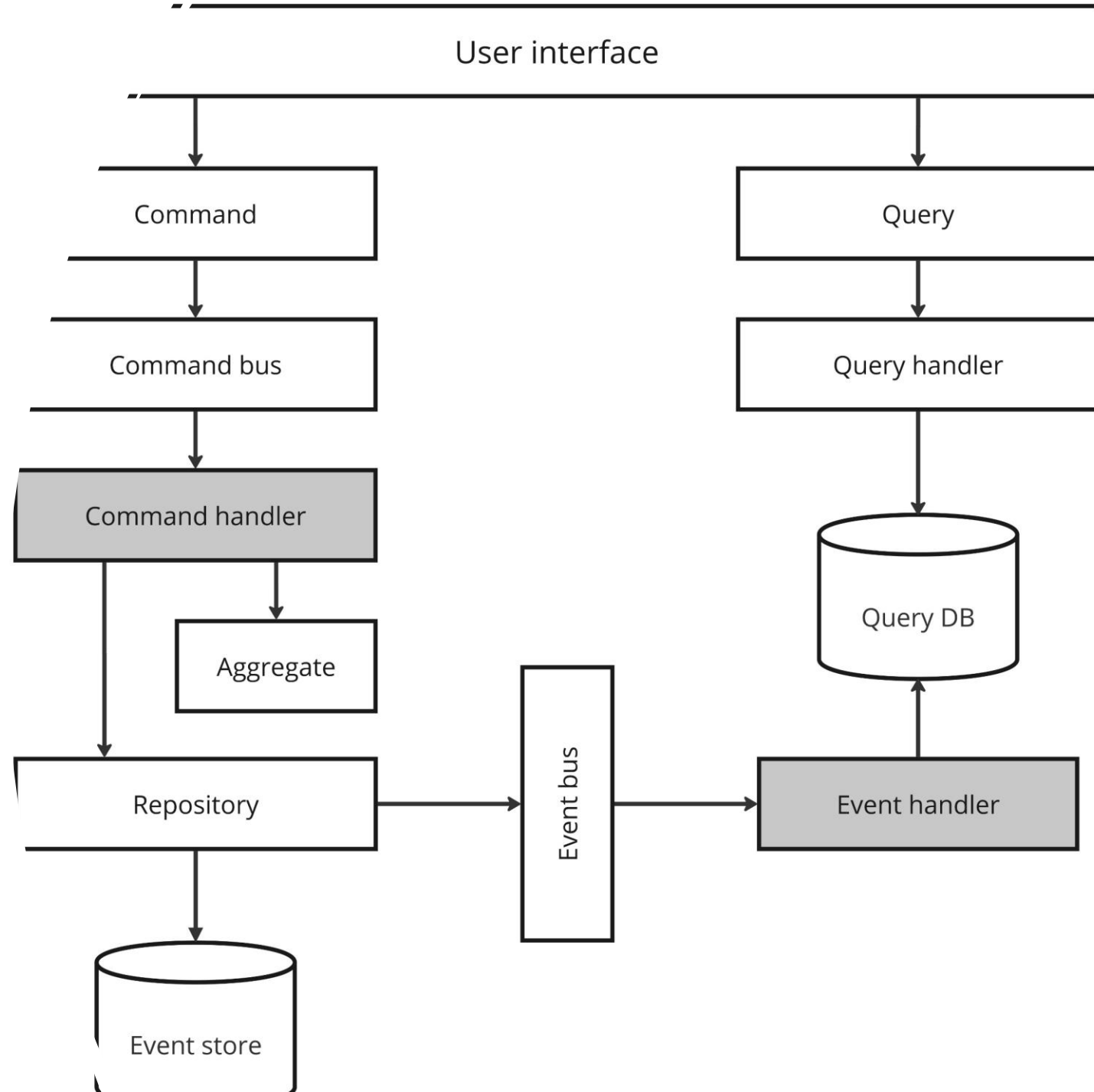
What exactly is a saga?

- A design pattern for managing distributed transactions across services.
- Ensures data consistency and **handles failures** with compensating transactions.
- Ideal for systems requiring complex transactions involving multiple services.
- Types
 - **Choreographed**: Services communicate via events, triggering actions without a central coordinator.
 - **Orchestrated**: A central service controls the sequence of actions.

Saga vs Process Manager

- Saga primarily focuses on ensuring data consistency while PM acts as a high-level coordinator that oversees the state and progression of a process.
- A PM retains state between messages while the saga does not. The saga decides what to do next based entirely on the content of the incoming message or event
- A saga is intended to help manage failures
- The logic handling canceling the process can be included inside of the process manager, and **this is why a lot of people threat sagas and process managers as the same thing** even though they are different being from the theoretical point of view (Seb Wilgosz)

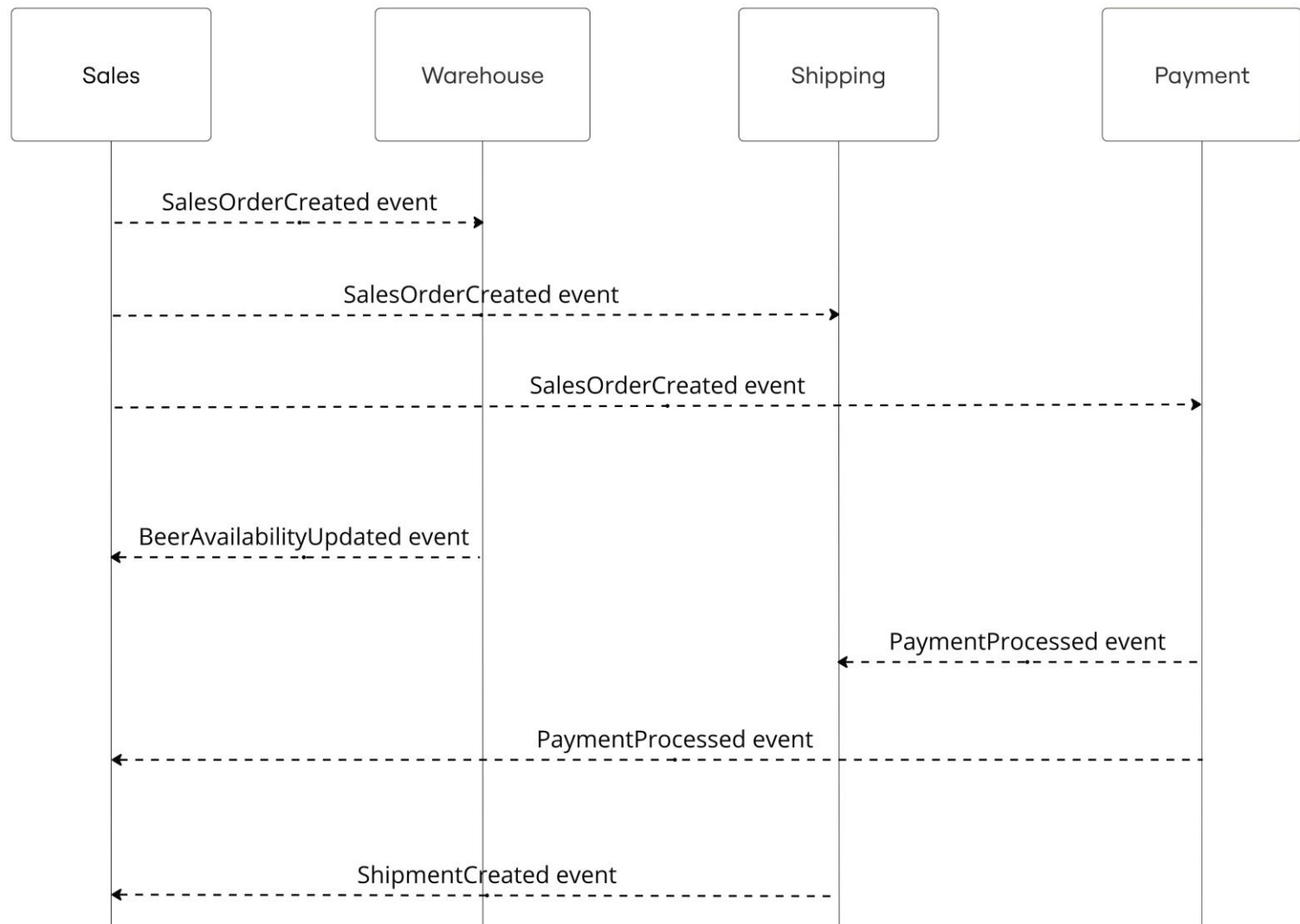
CQRS+ES
is not a
silver bullet!



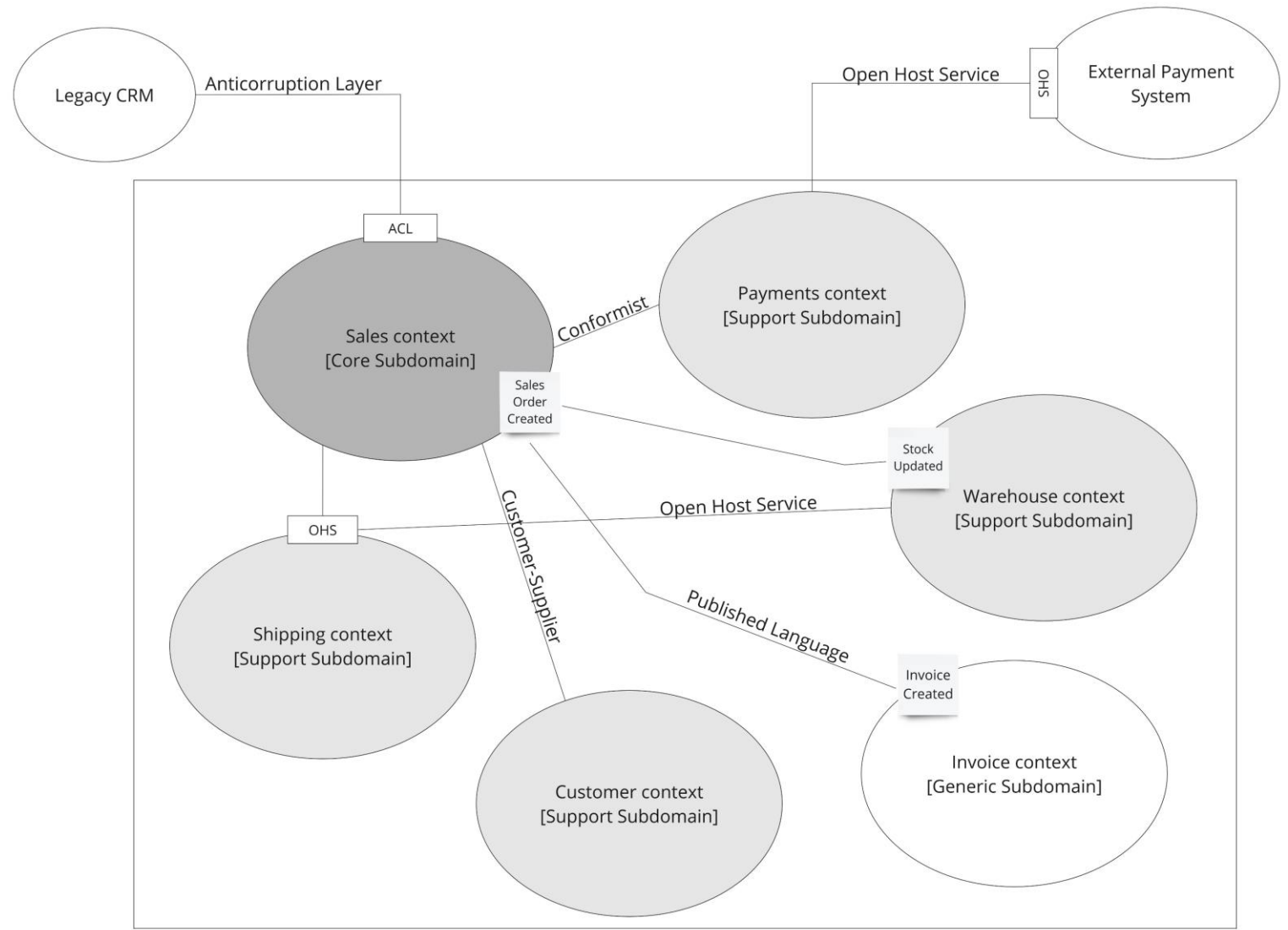
“Developers are drawn to complexity, like moths to a flame, often with the same outcome” - Neal Ford



The Beer ERP!



The Beer ERP!



How many sagas we can choose from?

Pattern Name	Communication	Consistency	Coordination
Epic Saga	Synchronous	Atomic	Orchestrated
Phone Tag Saga	Synchronous	Atomic	Choreographed
Fairy Tale Saga	Synchronous	Eventual	Orchestrated
Time Travel Saga	Synchronous	Eventual	Choreographed
Fantasy Fiction Saga	Asynchronous	Atomic	Orchestrated
Horror Story	Asynchronous	Atomic	Choreographed
Parallel Saga	Asynchronous	Eventual	Orchestrated
Anthology Saga	Asynchronous	Eventual	Choreographed

Epic Saga Pattern

Pros

Familiarity: Mimics monolithic systems, making it easier for architects familiar with traditional transactional models

Transactional coordination: Ensures all or nothing, maintaining atomic consistency across services

Clear ownership: The orchestrator manages and controls the workflow, providing a single point of coordination

Cons

High coupling: Exhibits strong coupling between services, increasing complexity in distributed systems

Performance bottlenecks: Synchronous communication and orchestration can lead to delays, impacting performance and scalability

Complex error handling: Compensating transactions introduce significant complexity, especially during failure scenarios

Limited scalability: The orchestrator's need to manage all transactions can make scaling and elasticity difficult.

Phone Tag Saga

Pros

Reduced bottlenecks: Lacks a central orchestrator, improving scalability and performance for successful workflows (happy paths).

Better scalability: Looser coupling and fewer wait conditions allow for higher throughput and scalability.

Improved responsiveness: Without a mediator, successful workflows can complete faster due to fewer coordination points.

Simplicity for basic workflows: Works well for simple workflows that have fewer error conditions.

Cons

Increased complexity: Each service must handle more workflow logic, including error handling and routing, making the pattern harder to manage.

Poor error handling: Without a central orchestrator, resolving errors requires complex compensating logic across services, leading to slower failure recovery.

Tight coupling remains: Despite improved scalability, services are still tightly coupled around transactional requirements, limiting scale in complex scenarios.

Limited suitability: Best suited for simple workflows; complexity grows quickly for more intricate workflows.

Fairy Tale Saga

Pros

Simpler error handling: Eventual consistency removes the need for strict transactional coordination, simplifying workflows.

Improved scalability: Services manage their own transactions, allowing independent scaling without tight coupling around atomicity.

Orchestration benefits: The orchestrator simplifies workflow management, making it easier to handle complex processes.

Low complexity: Combines orchestrated workflows and synchronous communication with fewer restrictions, offering a balanced trade-off for many use cases.

Cons

High coupling: Still retains high coupling in terms of synchronous communication and orchestration, which can limit flexibility.

Limited responsiveness: While more efficient than fully transactional sagas, synchronous communication can still impact performance and responsiveness.

Dependent on eventual consistency: Some systems might struggle with eventual consistency, especially for use cases requiring strict consistency.

Time Travel Saga

Pros

High scalability: Lack of an orchestrator and eventual consistency allow for easier scaling and independent service management.

Simpler transaction modeling: Each service owns its own transactionality, reducing the complexity of holistic transaction management.

Suitable for fast throughput: Well-suited for high-throughput, "fire and forget" workflows like data ingestion and bulk processing.

Reduced coupling: Without transactional coupling and orchestration, the pattern offers moderate decoupling, improving scalability.

Cons

Increased complexity for errors: Without a central orchestrator, error handling and workflow coordination must be managed by each service, adding complexity in failure scenarios.

Synchronous communication: Still relies on synchronous calls, which can slow responsiveness and performance, particularly in error conditions.

Limited to simple workflows: Best suited for simple, linear workflows; more complex workflows may require an orchestrator for better management.

Eventual consistency: Architects must ensure services synchronize data over time, which can complicate state management.

Fantasy Fiction Saga

Pros

Improved perceived performance: Asynchronous communication allows operations to happen in parallel, which can improve the perceived performance of workflows over synchronous communication (as in Epic Saga).

Maintains atomic consistency: This pattern ensures strong transactional guarantees, keeping workflows consistent throughout the entire process.

Cons

High coupling: The combination of atomicity and orchestrated coordination with asynchronous communication leads to extremely tight coupling, making it difficult to manage dependencies and out-of-order executions.

Increased complexity: Asynchronous workflows introduce design and debugging challenges, such as race conditions, deadlocks, and state management, making the system more difficult to scale and maintain.

Poor responsiveness and availability: Transactional coordination across asynchronous services negatively impacts responsiveness, especially when services are unavailable, and requires complex state tracking for pending transactions.

Limited scalability: Although asynchronous communication can improve responsiveness in non-transactional workflows, the transactional nature of this pattern makes it challenging to scale effectively, especially compared to eventual consistency approaches.

Horror Story Saga

Pros

Parallel work and scalability: The asynchronous communication aspect allows for tasks to be performed in parallel, improving throughput and enabling better scalability compared to synchronous workflows. The lack of a mediator also allows for more distributed coordination.

Cons

Horrific complexity: Combining atomic consistency with asynchronous communication and choreographed coordination creates the most complex scenario possible. Each service must track multiple pending transactions independently and out of order, leading to challenges like race conditions, undoing operations in the wrong sequence, and increased error management complexity.

Extremely difficult error handling: Without a mediator, services must handle errors themselves, including reversing operations that may have been executed out of order, making error recovery a nightmare.

Poor responsiveness: Despite the asynchronous nature, the need for each service to coordinate atomic transactions creates a lot of interservice communication ("chatter"). This hurts the overall system responsiveness, especially when errors occur.

Transactionality mismatch: The pattern tries to maintain atomic transactional integrity while operating in an asynchronous and choreographed environment, leading to significant coordination issues and high coupling around transactional states.

Parallel Saga

Pros

Low coupling: Services are more decoupled, allowing them to operate independently, without being blocked by other services.

Scalability and elasticity: Services can run in parallel, with each service handling its own transactions, making it easy to scale parts of the system independently.

Good performance: Services don't need to wait for others to complete their tasks, reducing bottlenecks and improving overall system throughput.

Cons

Complexity in synchronization: Asynchronous communication introduces challenges like race conditions, deadlocks, and difficulties in ensuring data synchronization.

Eventual consistency: Data may not be consistent in real-time. This introduces complexities in handling shared state and data synchronization across services.

Increased burden on mediator: The lack of transactional boundaries between services increases the responsibility of the orchestrator, particularly in managing all scenarios for error recovery, retries, and compensating transactions.

Anthology Saga (Asynchronous - Eventual - Coreographed)

Pro

Low coupling: With no orchestrator and asynchronous communication, each service operates independently.

High scalability and elasticity: Asynchronous communication and eventual consistency allow each service to process tasks in parallel, which improves the system's ability to scale and adapt to varying loads.

High throughput and performance: optimized for systems that need high responsiveness and throughput. Since there is no orchestration and no global transactions, there are fewer bottlenecks, which leads to faster overall performance.

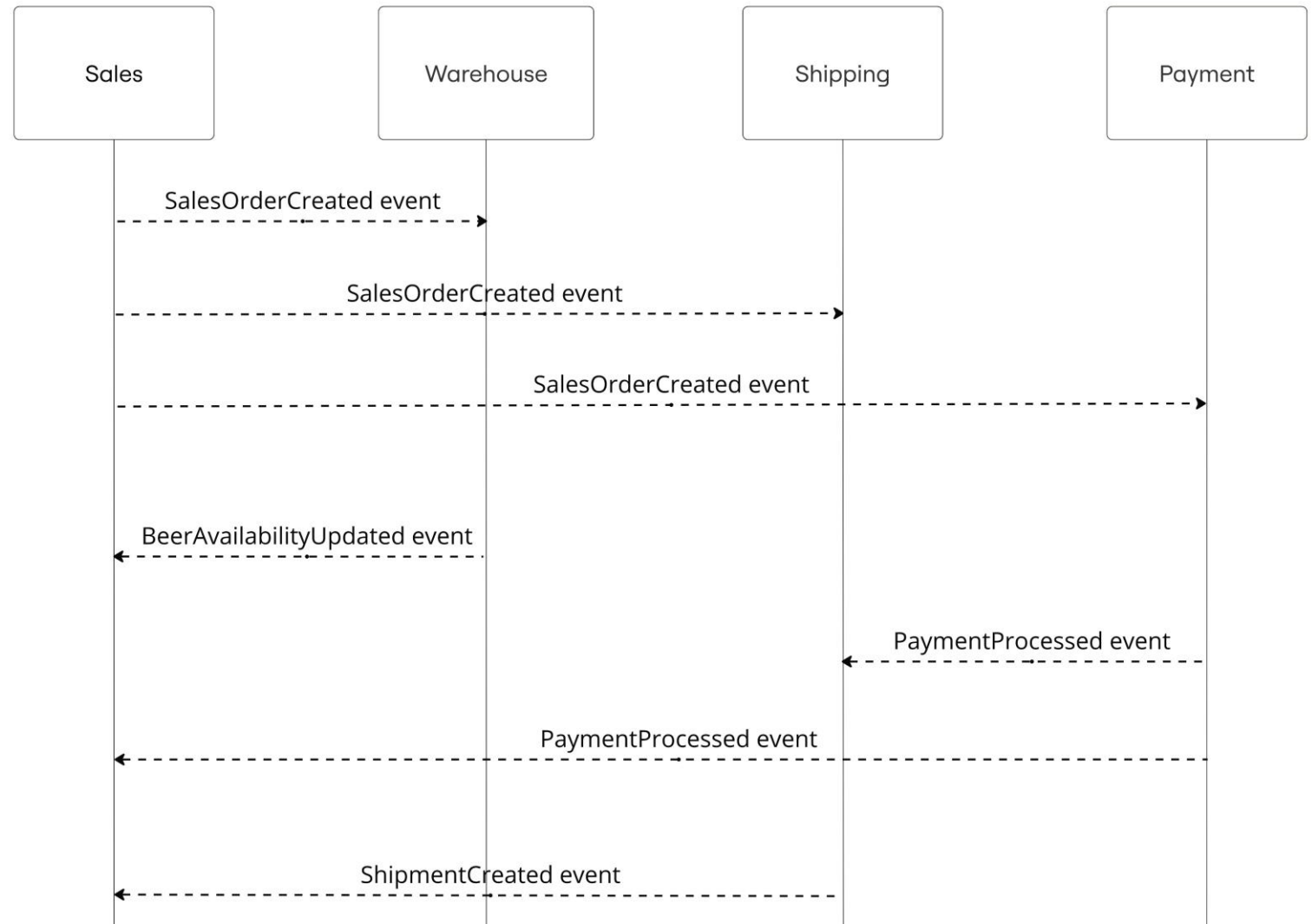
Cons

High complexity for workflows: The lack of an orchestrator adds complexity to the design, especially for complex workflows.

Challenging error handling: Services need to be designed with robust error-handling and coordination strategies to ensure eventual consistency.

Not ideal for complex workflows: This pattern is best suited for simple or mostly linear workflows. For more intricate or multi-step workflows, the lack of centralized coordination can make managing workflow state across services challenging.

The Beer ERP!



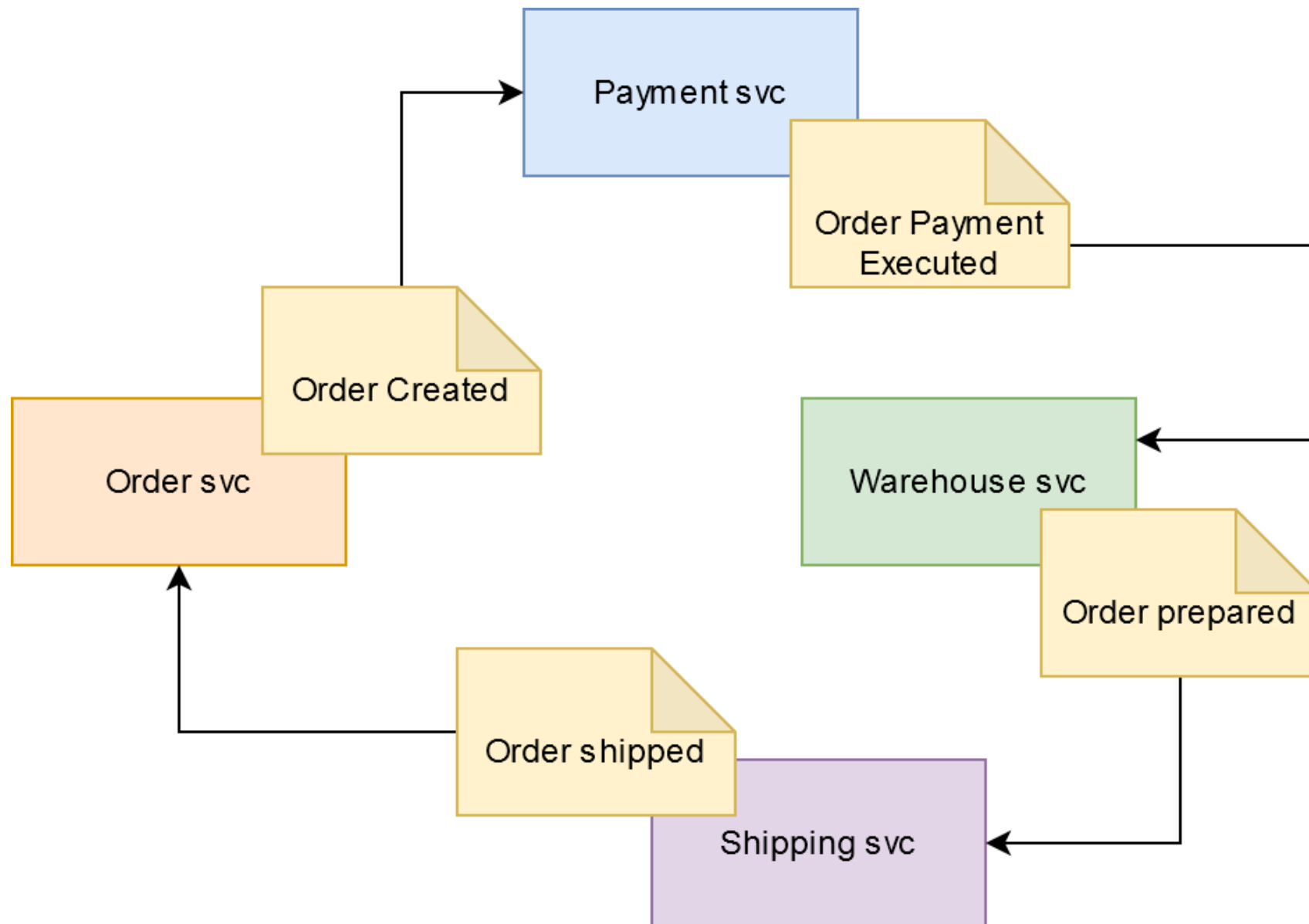
A blurred background image featuring a laptop screen with code and a pen holder on the left. The code on the screen includes functions like 'ratingPromptCheck', 'ratingPromptTimer', and 'displayRatingPrompt'.

Exercise

Design an Anthology Saga

Anthology Saga

Asynchronous - Eventual - Coreographed

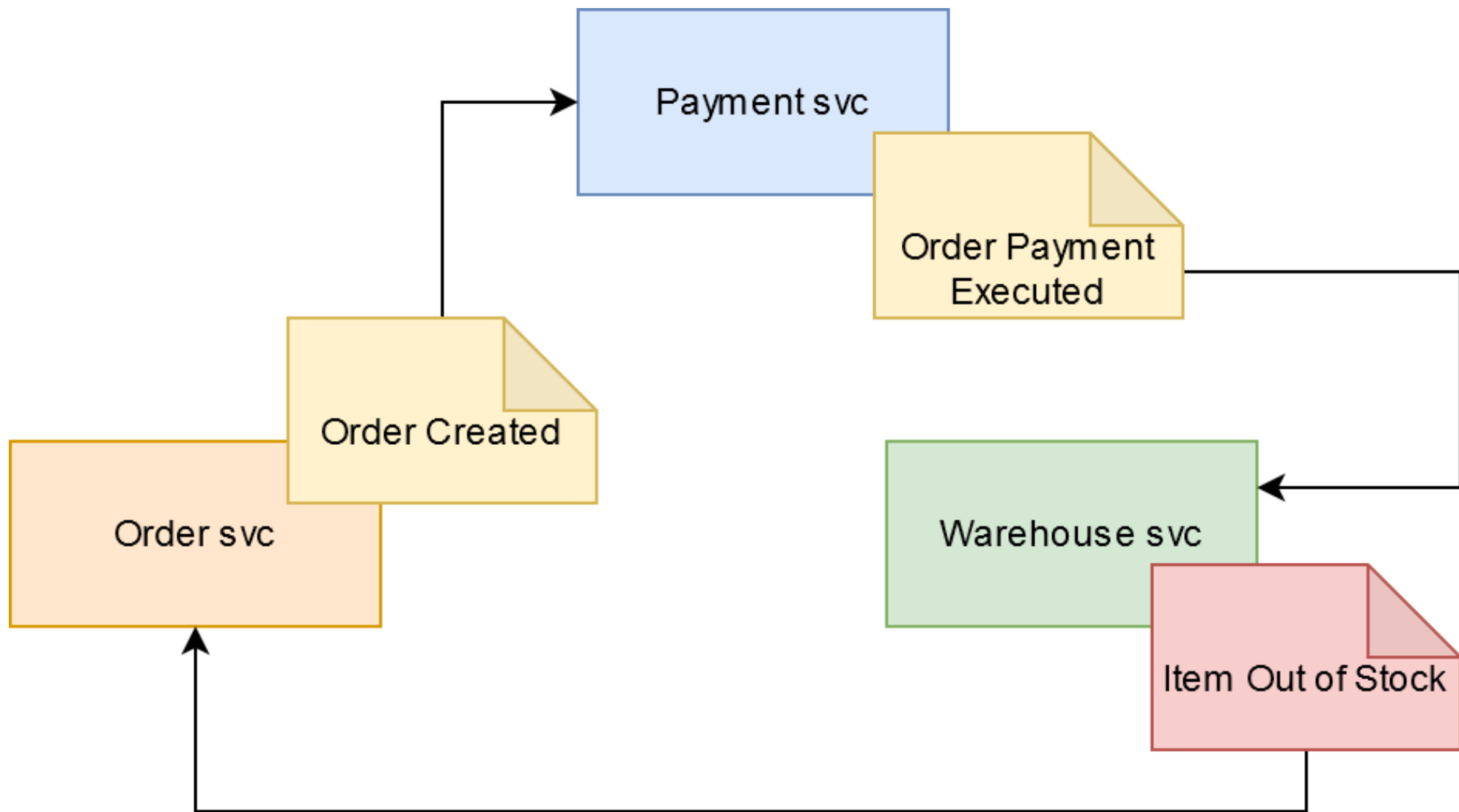


Anthology Saga

- There is no central coordinator
- Each service produces and listens to events coming from other services
- It ends when the last service completes its transaction and no longer emits events
- Decoupled components
- What if I wanted to update the order status as it progresses?

What if the product is not in stock?

What is the cost of rolling back a distributed transaction?



Rollback cost

- Order service and payment service also listen to the "Item out of stock/order not prepared" event.
- Payment service processes the refund.
- Order service sets the status to "cancelled".
- You see where we're going with a complex transaction...

A blurred background image of a laptop screen showing code, with a pen holder containing several pens in the foreground on the left. The text 'Show me the code!' is overlaid in white.

Show me the code!

Anthology Saga

Parallel Saga (Asynchronous - Eventual - Orchestrated)

Pros

Low coupling: Services are more decoupled, allowing them to operate independently, without being blocked by other services.

Scalability and elasticity: Services can run in parallel, with each service handling its own transactions, making it easy to scale parts of the system independently.

Good performance: Services don't need to wait for others to complete their tasks, reducing bottlenecks and improving overall system throughput.

Cons

Complexity in synchronization: Asynchronous communication introduces challenges like race conditions, deadlocks, and difficulties in ensuring data synchronization.

Eventual consistency: Data may not be consistent in real-time. This introduces complexities in handling shared state and data synchronization across services.

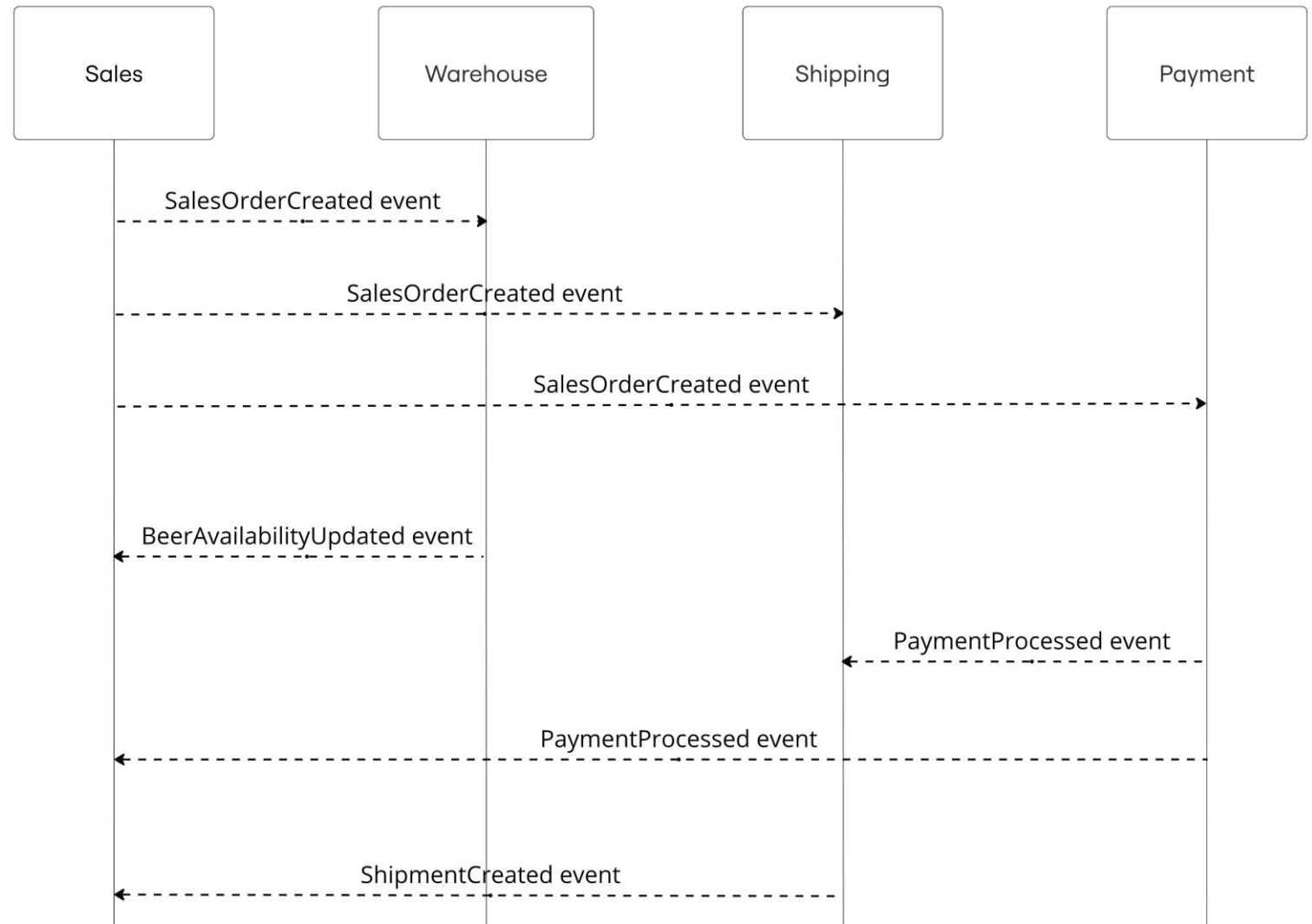
Increased burden on mediator: The lack of transactional boundaries between services increases the responsibility of the orchestrator, particularly in managing all scenarios for error recovery, retries, and compensating transactions.

A blurred background image of a workspace. On the left, a brown mesh pen holder contains several pens and pencils. In the center, a laptop screen displays code in a dark-themed editor. The code includes comments in Chinese and JavaScript/TypeScript for a rating prompt system, using 'Cache' for state management and 'setTimeout' for timing. The word 'Exercise' is overlaid in large white text.

Exercise

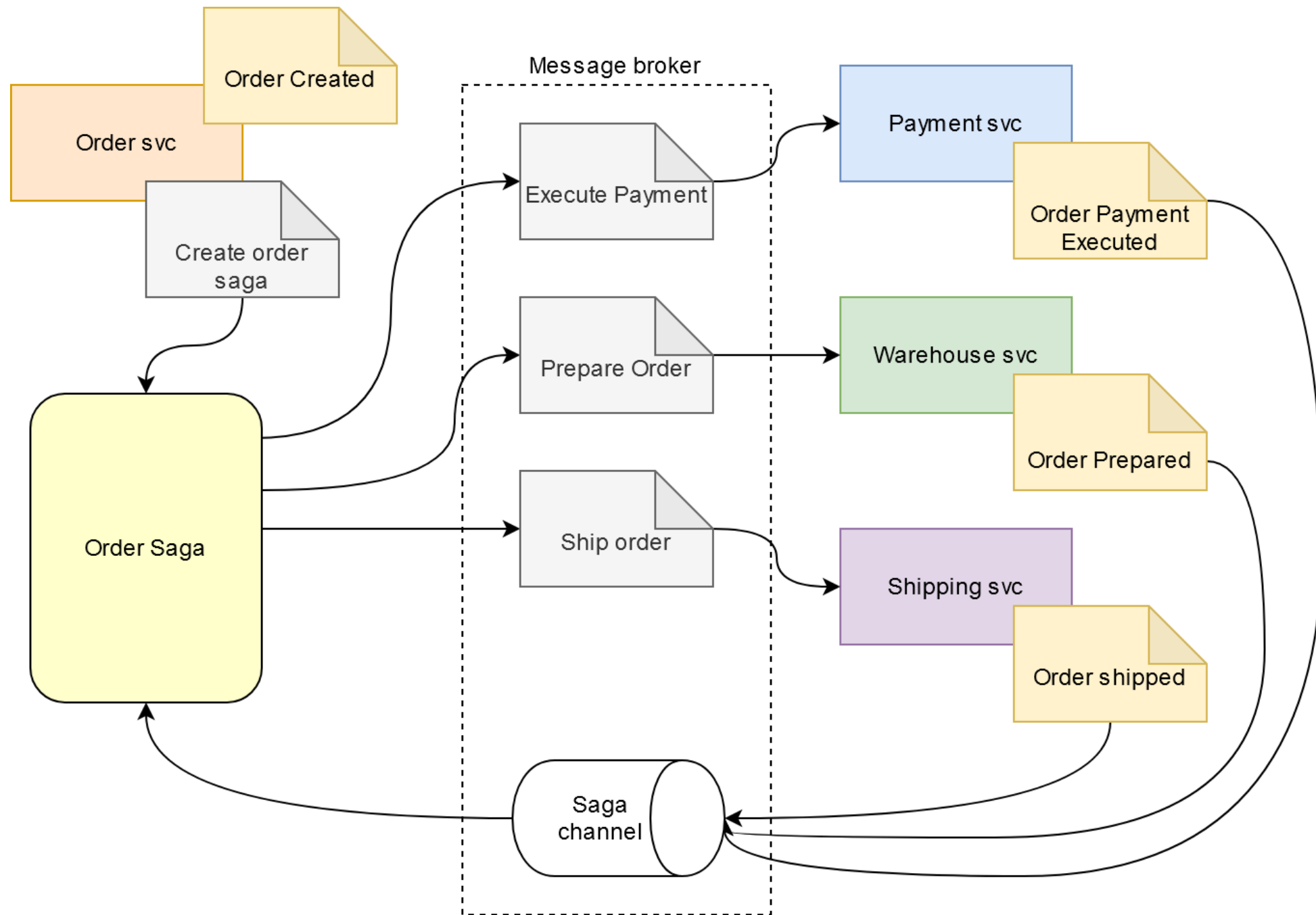
Design a Parallel Saga

The Beer ERP!



Parallel Saga

Asynchronous - Eventual - Coreographed

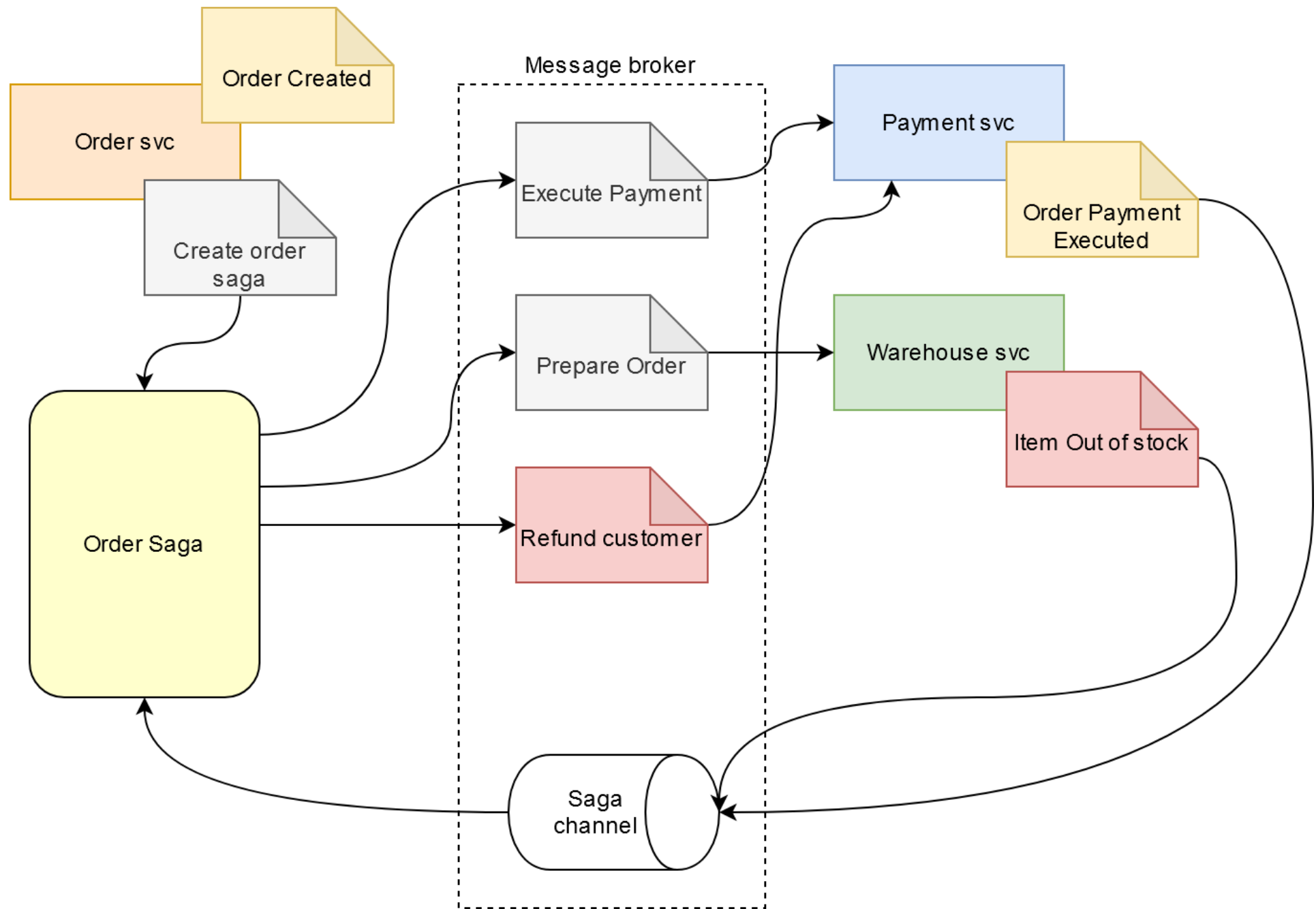


Parallel Saga

- We will have a service responsible for issuing commands and handling events
- It all starts with a **command**
- A state machine fits well here

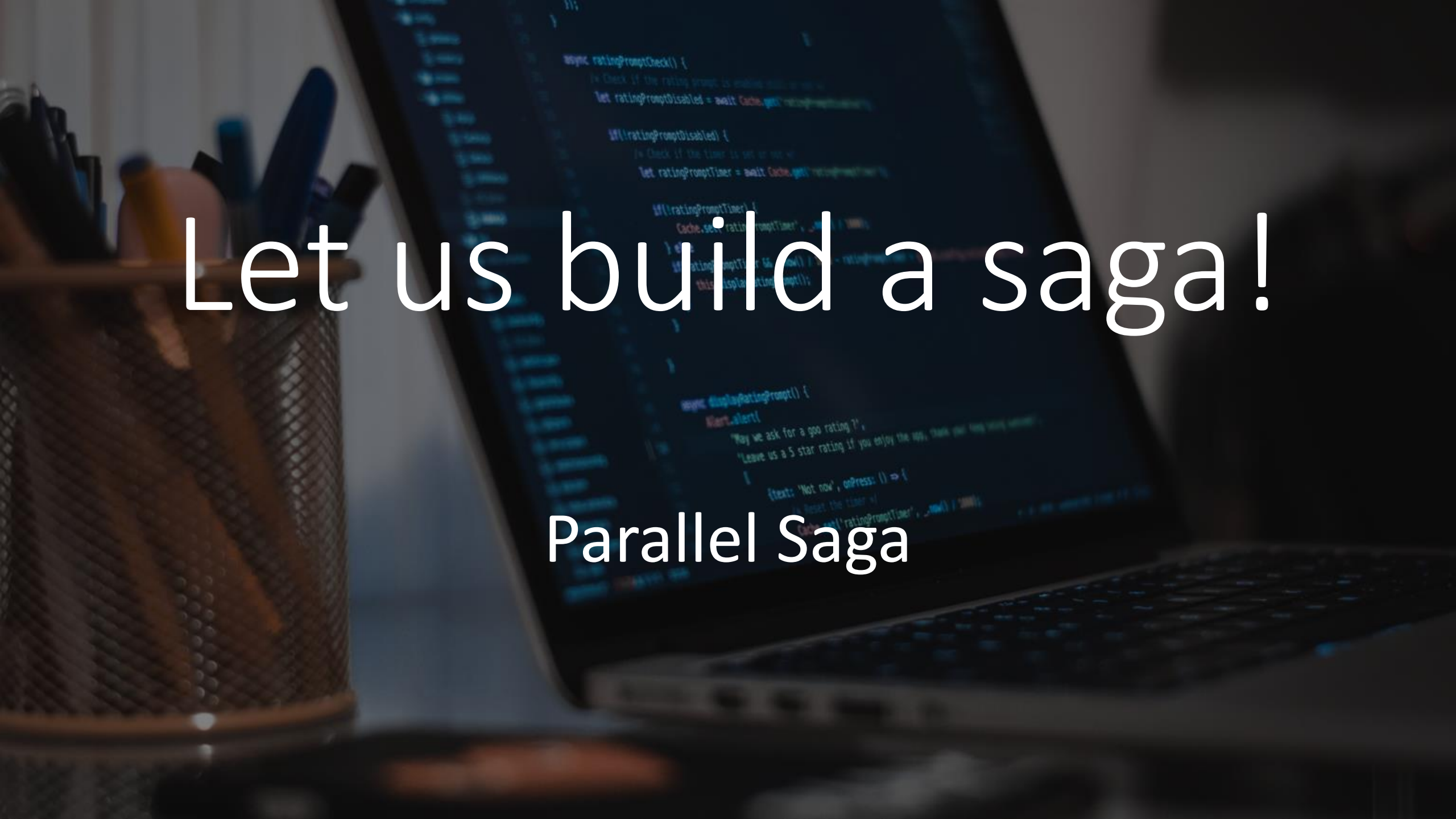
What if the product is not in stock?

What is the cost of rolling back a distributed transaction?



Rollback costs

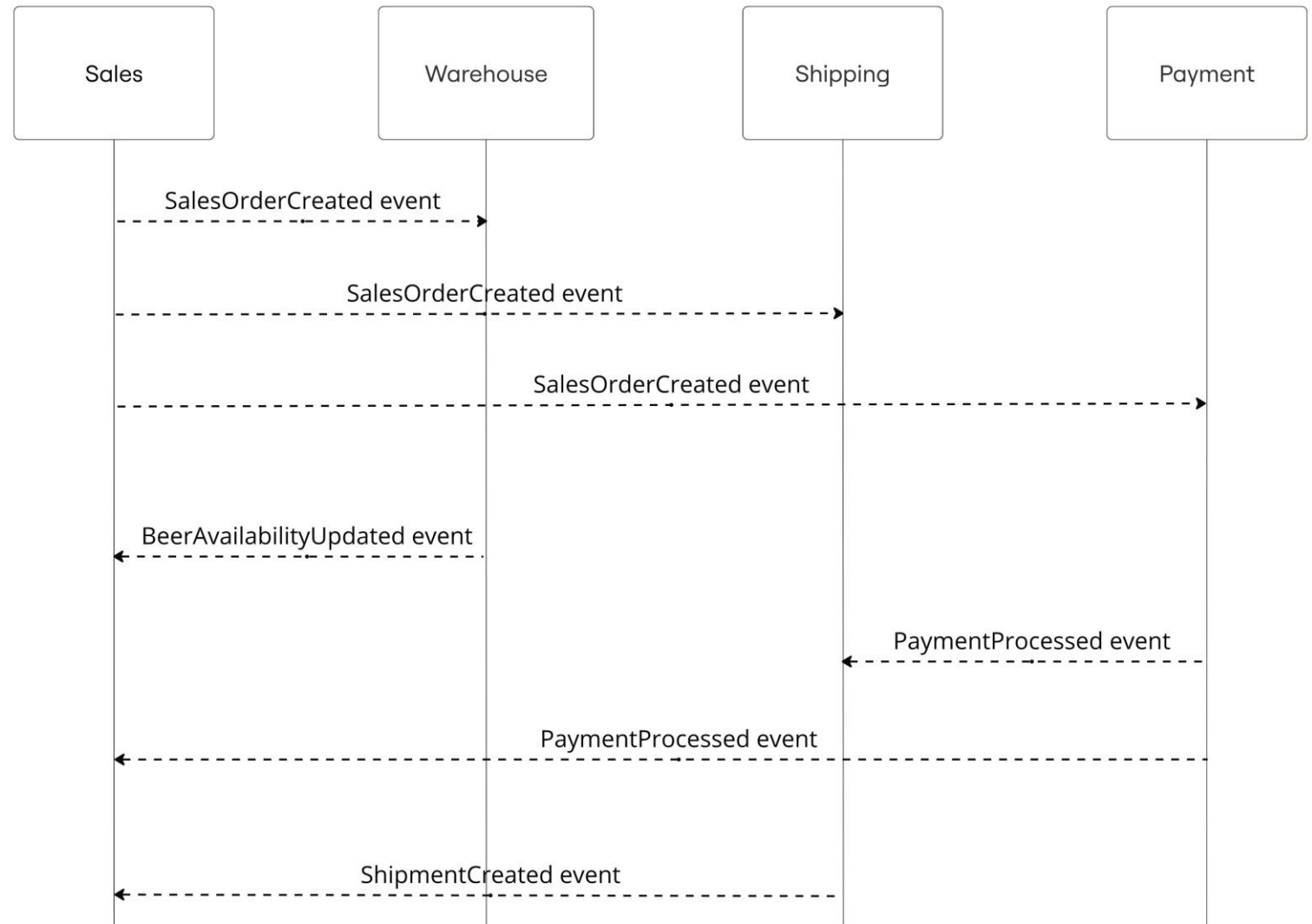
- Easier than Choreography
- It must listen to all “anomalies” events
- All rollback steps are managed by the orchestrator.

A blurred background image of a workspace. On the left, a metal mesh pen holder contains several pens and pencils. In the center, a laptop screen displays a code editor with syntax-highlighted code. The code includes comments in Chinese and JavaScript-like function calls such as `Cache.get`, `ratingPromptTimer`, and `displayRatingPrompt`.

Let us build a saga!

Parallel Saga

The Beer ERP!



How do you test a Saga?

With specification testing!

Given, When, Then

- The essential idea is to divide a scenario into three sections:
- **Given**: describes the state of the aggregate before sending the command. A kind of prerequisite of the test.
- **When**: represents the command to be sent to the aggregate.
- **Then/Expect**: describes the state changes expected as a result of the command.



A photograph of a laptop screen displaying code, with a pen holder in the foreground. The text 'Show me the code!' is overlaid in white. The code on the screen is as follows:

```
11;
12
13 async ratingPromptCheck() {
14   // Check if the rating prompt is enabled on the server
15   let ratingPromptDisabled = await Cache.get('ratingPromptDisabled');
16
17   if(!ratingPromptDisabled) {
18     // Check if the timer is set or not yet
19     let ratingPromptTimer = await Cache.get('ratingPromptTimer');
20
21     if(!ratingPromptTimer) {
22       Cache.set('ratingPromptTimer', new Date().getTime() / 1000);
23       // If the timer is set, then the rating prompt is shown
24       if(ratingPromptTimer < new Date().getTime() / 1000) {
25         this.displayRatingPrompt();
26       }
27     }
28   }
29
30   async displayRatingPrompt() {
31     Alert.alert(
32       'May we ask for a goo rating?',
33       'Leave us a 5 star rating if you enjoy the app. Thank you! Keep using software!',
34       [
35         {text: 'Not now', onPress: () => {
36           // Reset the timer
37           Cache.set('ratingPromptTimer', new Date().getTime() / 1000);
38         }},
39       ],
40     );
41   }
42 }
```

Bonus

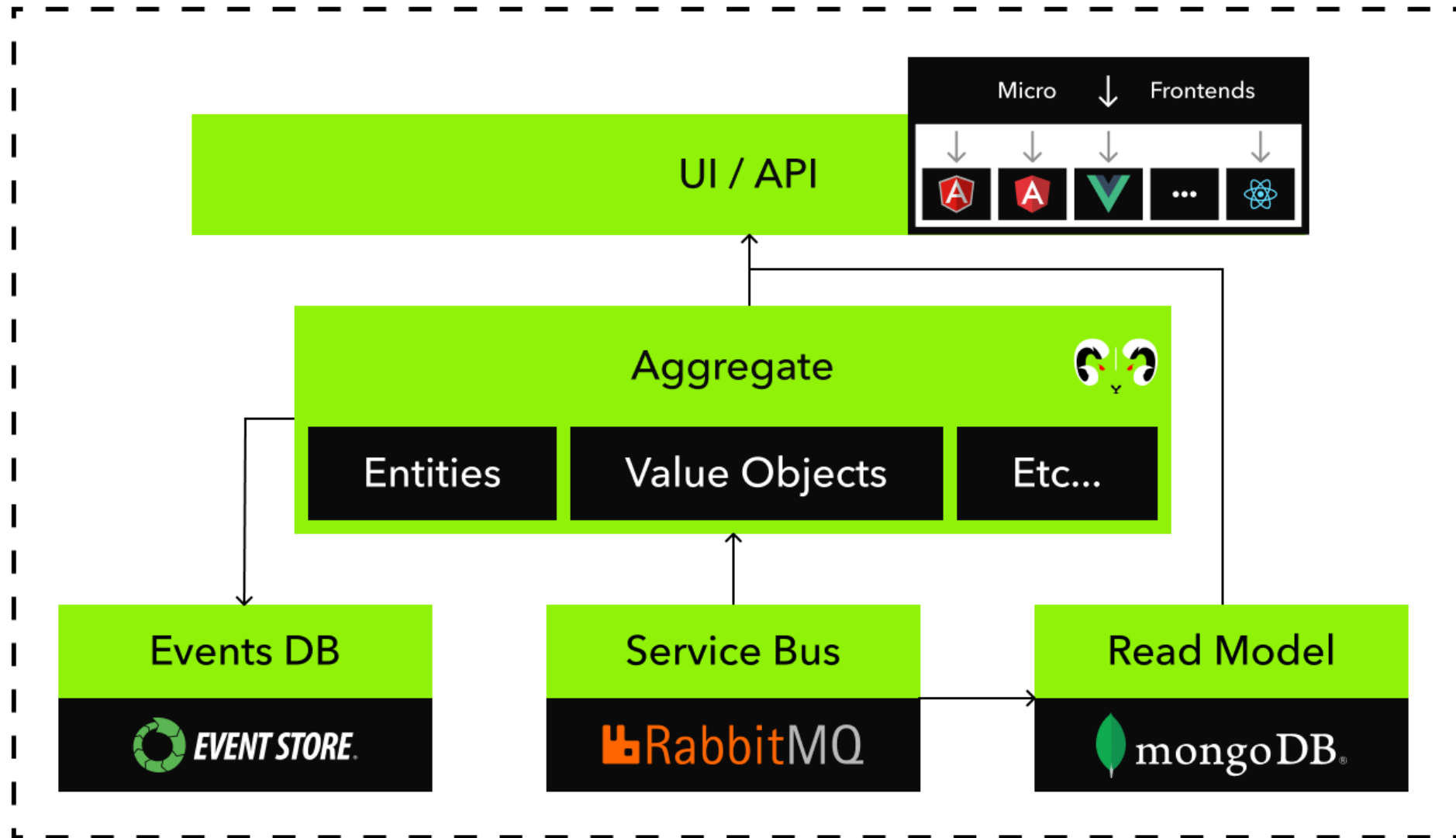
“Software architecture it's abstract by nature and we must ground it with some implementation details to make it concrete”

Architecture quantum | quanta

An architecture quantum measures several aspects of both topology and behavior in software architecture related to how parts connect and communicate with one another.

It is an independently deployable artifact with high functional cohesion, high static coupling, and synchronous dynamic coupling.

Components Cohesion / Quantum Arch.



Last responsibility moment

«Delay decisions as long as you can, but not longer. Maximize the information you have. Minimize technical debt from complexity.»

Postel's law or Robustness principle

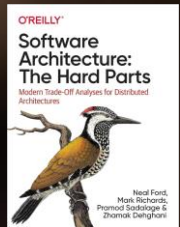
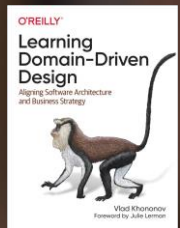
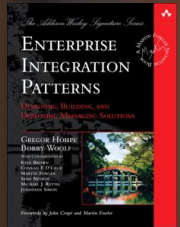
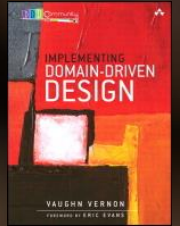
*“Be conservative in what you do, be liberal in what you accept from others”
Only validate what you need*

Trade-off everywhere

*«Don't try to find the best design in software architecture.
Instead, strive for the least worst combination of trade-offs»*

Bibliography

- **Implementing Domain-Driven Design (V. Vernon)**
<https://www.amazon.it/Implementing-Domain-Driven-Design-Vaughn-Vernon-ebook/dp/B00BCLEBN8/>
- **REST in Practice: Hypermedia and Systems Architecture (J. Webber)**
<https://www.amazon.it/REST-Practice-Hypermedia-Systems-Architecture/dp/0596805829>
- **Enterprise Integration Patterns (G. Hohpe)**
<https://www.amazon.it/Enterprise-Integration-Patterns-Designing-Deploying/dp/0321200683/>
- **Learning Domain-Driven Design: Aligning Software Architecture and Business Strategy (Vlad Khononov)**
<https://www.amazon.com/Learning-Domain-Driven-Design-Aligning-Architecture/dp/1098100131>
- **Hands-On Domain-Driven Design with .NET Core (A. Zimarev)**
<https://www.amazon.it/Hands-Domain-Driven-Design-NET/dp/1788834097>
- **Software Architecture: The Hard Parts (N. Ford, M. Richards, P. Sadalage, Z. Dehghani)**
<https://www.amazon.com/Software-Architecture-Trade-Off-Distributed-Architectures/dp/1492086894>



Thank
You

You can find us at

✕ @aacerbis

 <https://www.linkedin.com/in/aacerbis/>

 alberto.acerbis@intre.it



✕ @collaalessandro

 <https://www.linkedin.com/in/alessandrocolla>

 alessandro.colla@evoluzione.agency

